Watermarking Strategies for Late Data

Prepared by : Kaviraj T U

Handling Out-of-Order Events in Spark Streaming

In real-time data pipelines, **events rarely arrive in perfect order**.

Network delays, retries, mobile connectivity, and distributed producers often cause **late or out-of-order events**.

If not handled properly, these events can **corrupt aggregations, produce incorrect metrics, or grow state indefinitely**.

Spark Structured Streaming solves this using **Watermarking**.

The Problem: Late and Out-of-Order Data

Imagine a streaming pipeline processing **user transactions**.

Events arrive with timestamps:

```
10:00:01
10:00:03
10:00:02  ← late event
10:00:07
```

Spark processes data in **micro-batches**, but event timestamps may not follow processing order.

Without special handling:

- Aggregations may be incorrect

• Windows may close too early

• State may grow infinitely

Watermarking introduces a **controlled delay tolerance**.

What is Watermarking?

A **watermark** tells Spark:

> "Accept late data up to a certain delay, then finalize the result."

Example watermark:

```
10 minutes
```

Meaning:

If an event arrives **within 10 minutes of its event time**, it will still be processed.

Events arriving **after the watermark threshold** are ignored.

Example in Spark

```
df \
.withWatermark("event_time","10 minutes") \
.groupBy(
window("event_time","5 minutes"),
"user_id"
```

```
  ) \
.count()
```

This means:

- Window size = **5 minutes**

• Late events allowed for **10 minutes**

Spark will keep window state until the watermark passes.

How Watermarks Work Internally

Spark tracks the **maximum event time observed**.

Watermark =

```
max_event_time - watermark_delay
```

Example:

```
Max event seen → 10:30
Watermark delay → 10 minutes
Watermark time → 10:20
```

Events earlier than **10:20** are considered **too late** and discarded.

Why Watermarking Matters

Without watermarks:

- Streaming state grows indefinitely

- Memory usage increases
- Streaming jobs eventually fail

Watermarks allow Spark to **clean up old state safely**.

This is critical for **long-running streaming pipelines**.

Common Watermark Strategies

1️⃣ Fixed Delay Watermark

Most common approach.

Example:

```
10 minutes
```

Best when late events are predictable.

2️⃣ Large Safety Window

For systems where events may arrive very late.

Example:

```
1 hour watermark
```

Trade-off:

✔️ More accurate results

❌ Larger state size

3️⃣ Event-Time Based Windowing

Use event timestamps instead of processing time.

This ensures correctness when events arrive late.

Risks of Incorrect Watermark Configuration

If watermark delay is **too small**:

- Late events are dropped

• Aggregations become inaccurate

If watermark delay is **too large**:

- State grows significantly

• Higher memory usage

• Slower streaming queries

Choosing the correct delay requires **understanding your data arrival patterns**.

Real Production Insight

Late data is extremely common in:

- IoT pipelines

• Mobile analytics

• Clickstream processing

• Distributed event systems

Without watermarking, streaming systems often become **unstable over time**.

A well-designed watermark strategy ensures both:

✔️ Accurate analytics

✔️ Controlled state growth

Final Takeaway

In real-time data pipelines, **perfect event ordering is unrealistic**.

Watermarking allows Spark to balance:

- Accuracy
- Latency
- Resource usage

Handling late events effectively is a **key skill in building reliable streaming systems**.

Thank you.!

Happy Learning...